

```
CREATE DATABASE banking_db;
```

```
USE banking_db;
```

```
CREATE TABLE Branches
```

```
(  
    branch_id VARCHAR(10) PRIMARY KEY,  
    branch_name VARCHAR(100),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Customers
```

```
(  
    customer_id VARCHAR(10) PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Accounts
```

```
(  
    account_no VARCHAR(10) PRIMARY KEY,  
    customer_id VARCHAR(10),  
    branch_id VARCHAR(10),  
    account_type VARCHAR(30),  
    balance DECIMAL(15,2),
```

```
FOREIGN KEY(customer_id)  
    REFERENCES Customers(customer_id),
```

```
FOREIGN KEY(branch_id)
REFERENCES Branches(branch_id)
);
```

```
CREATE TABLE Transactions
(
transaction_id VARCHAR(10) PRIMARY KEY,
account_no VARCHAR(10),
transaction_type VARCHAR(30),
amount DECIMAL(15,2),
transaction_date DATE,

FOREIGN KEY(account_no)
REFERENCES Accounts(account_no)
);
```

```
CREATE TABLE Loans
(
loan_id VARCHAR(10) PRIMARY KEY,
customer_id VARCHAR(10),
loan_type VARCHAR(50),
loan_amount DECIMAL(15,2),

FOREIGN KEY(customer_id)
REFERENCES Customers(customer_id)
);
```

```
CREATE TABLE Loan_Payments
(
```

```
payment_id VARCHAR(10) PRIMARY KEY,  
loan_id VARCHAR(10),  
payment_amount DECIMAL(15,2),  
payment_date DATE,  
  
FOREIGN KEY(loan_id)  
REFERENCES Loans(loan_id)  
);
```

```
INSERT INTO Branches (branch_id,branch_name,city) VALUES  
( 'BR001','Chennai Main','Chennai'),  
( 'BR002','Anna Nagar','Chennai'),  
( 'BR003','Bangalore  
Central','Bangalore'),  
( 'BR004','Hyderabad City','Hyderabad'),  
( 'BR005','Mumbai West','Mumbai');
```

```
INSERT INTO Customers (customer_id,customer_name,city) VALUES  
( 'CUS001','Ahmed','Chennai'),  
( 'CUS002','John','Bangalore'),  
( 'CUS003','Sarah','Hyderabad'),  
( 'CUS004','David','Mumbai'),  
( 'CUS005','Priya','Chennai'),  
( 'CUS006','Rahul','Coimbatore'),  
( 'CUS007','Fatima','Delhi'),  
( 'CUS008','Meena','Salem'),  
( 'CUS009','Kumar','Madurai'),  
( 'CUS010','Anita','Pune'),  
( 'CUS011','Ravi','Trichy'),
```

```
('CUS012','Ayesha','Kochi');
```

```
INSERT INTO
```

```
Accounts(account_no,customer_id,branch_id,account_type,balance) VALUES
```

```
('ACC1001','CUS001','BR001','Savings',85000),
```

```
('ACC1002','CUS002','BR003','Savings',45000),
```

```
('ACC1003','CUS003','BR004','Current',120000),
```

```
('ACC1004','CUS004','BR005','Savings',65000),
```

```
('ACC1005','CUS005','BR002','Savings',98000),
```

```
('ACC1006','CUS006','BR001','Current',150000),
```

```
('ACC1007','CUS007','BR004','Savings',40000),
```

```
('ACC1008','CUS008','BR001','Savings',76000),
```

```
('ACC1009','CUS009','BR002','Current',110000),
```

```
('ACC1010','CUS010','BR003','Savings',53000),
```

```
('ACC1011','CUS011','BR005','Savings',30000),
```

```
('ACC1012','CUS012','BR004','Current',170000);
```

```
INSERT INTO
```

```
Transactions(transaction_id,account_no,transaction_type,amount,transaction_date) VALUES
```

```
('TX001','ACC1001','Deposit',10000,'2025-01-02'),
```

```
('TX002','ACC1001','Withdrawal',5000,'2025-01-05'),
```

```
('TX003','ACC1002','Deposit',20000,'2025-01-07'),
```

```
('TX004','ACC1003','Withdrawal',15000,'2025-01-10'),
```

```
('TX005','ACC1004','Deposit',8000,'2025-01-12'),
```

```
('TX006','ACC1005','Withdrawal',3000,'2025-01-15'),
```

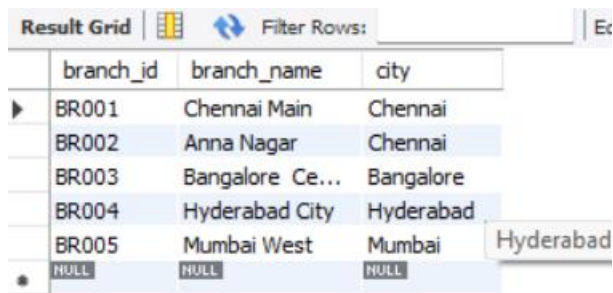
```
('TX007','ACC1006','Deposit',25000,'2025-01-18'),
```

```
('TX008','ACC1007','Deposit',12000,'2025-01-20'),  
('TX009','ACC1008','Withdrawal',7000,'2025-01-22'),  
('TX010','ACC1009','Deposit',50000,'2025-01-25');
```

```
INSERT INTO Loans(loan_id,customer_id,loan_type,loan_amount) VALUES  
('LN001','CUS001','Home Loan',2500000),  
('LN002','CUS003','Car Loan',800000),  
('LN003','CUS005','Education Loan',600000),  
('LN004','CUS007','Personal Loan',300000),  
('LN005','CUS010','Business Loan',1500000),  
('LN006','CUS012','Home Loan',3200000);
```

```
INSERT INTO  
Loan_Payments(payment_id,loan_id,payment_amount,payment_date)  
VALUES  
('PAY001','LN001',50000,'2025-01-15'),  
('PAY002','LN001',50000,'2025-02-15'),  
('PAY003','LN002',25000,'2025-01-20'),  
('PAY004','LN003',20000,'2025-01-25'),  
('PAY005','LN004',15000,'2025-01-28'),  
('PAY006','LN005',60000,'2025-02-02'),  
('PAY007','LN006',70000,'2025-02-05'),  
('PAY008','LN006',70000,'2025-03-05');
```

SELECT * FROM Branches;

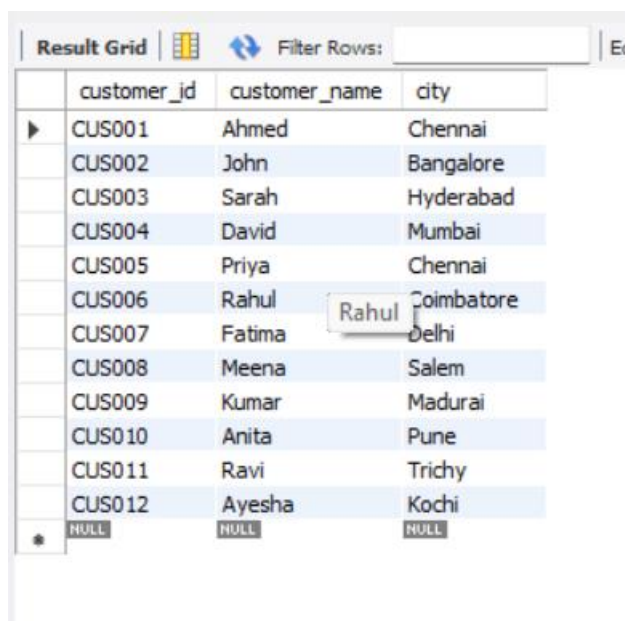


The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains a table with 4 columns: 'branch_id', 'branch_name', and 'city'. The data is as follows:

branch_id	branch_name	city
BR001	Chennai Main	Chennai
BR002	Anna Nagar	Chennai
BR003	Bangalore Ce...	Bangalore
BR004	Hyderabad City	Hyderabad
BR005	Mumbai West	Mumbai
NULL	NULL	NULL

A mouse cursor is hovering over the 'Hyderabad' cell in the 'city' column of the row with 'BR004'.

SELECT * FROM Customers;



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains a table with 4 columns: 'customer_id', 'customer_name', and 'city'. The data is as follows:

customer_id	customer_name	city
CUS001	Ahmed	Chennai
CUS002	John	Bangalore
CUS003	Sarah	Hyderabad
CUS004	David	Mumbai
CUS005	Priya	Chennai
CUS006	Rahul	Coimbatore
CUS007	Fatima	Delhi
CUS008	Meena	Salem
CUS009	Kumar	Madurai
CUS010	Anita	Pune
CUS011	Ravi	Trichy
CUS012	Ayesha	Kochi
NULL	NULL	NULL

A mouse cursor is hovering over the 'Rahul' cell in the 'customer_name' column of the row with 'CUS006'.

SELECT * FROM Accounts;

Result Grid			 Filter Rows:	Edit:   	Exp
	account_no	customer_id	branch_id	account_type	balance
▶	ACC1001	CUS001	BR001	Savings	85000.00
	ACC1002	CUS002	BR003	Savings	45000.00
	ACC1003	CUS003	BR004	Current	120000.00
	ACC1004	CUS004	BR005	Savings	65000.00
	ACC1005	CUS005	BR002	Savings	98000.00
	ACC1006	CUS006	BR001	Current	150000.00
	ACC1007	CUS007	BR004	Savings	40000.00
	ACC1008	CUS008	BR001	Savings	76000.00
	ACC1009	CUS009	BR002	Current	110000.00
	ACC1010	CUS010	BR003	Savings	53000.00
	ACC1011	CUS011	BR005	Sa Savings	30000.00
	ACC1012	CUS012	BR004	Current	170000.00
•	NULL	NULL	NULL	NULL	NULL

SELECT * FROM Transactions;

Result Grid

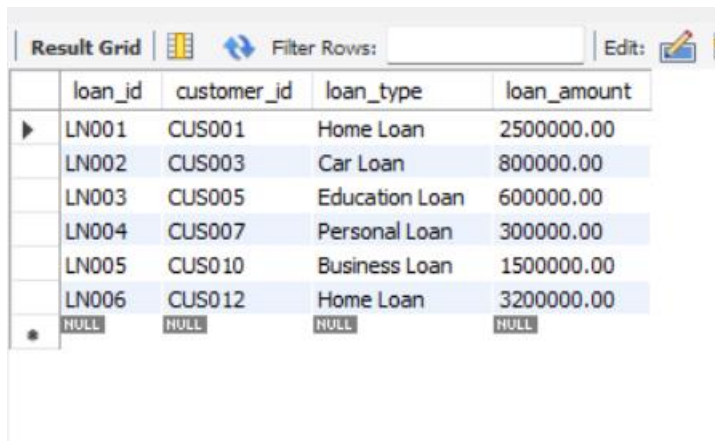
Filter Rows:

Edit:

Export/Imp

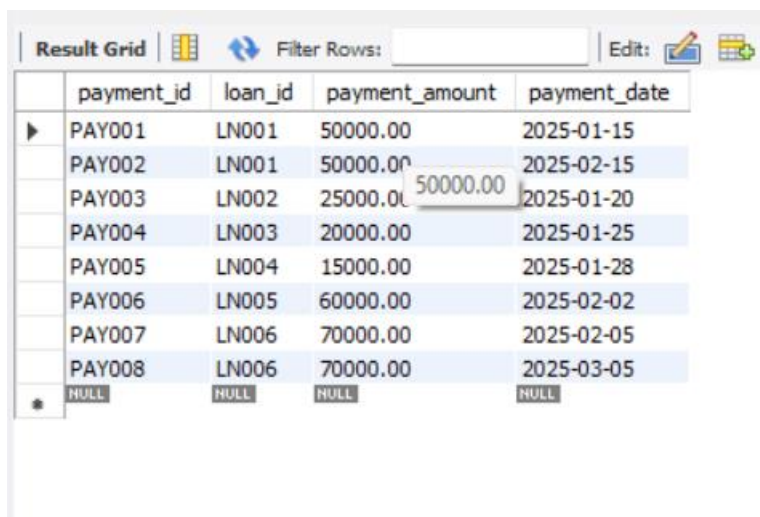
	transaction_id	account_no	transaction_type	amount	transaction_date
▶	TX001	ACC1001	Deposit	10000.00	2025-01-02
	TX002	ACC1001	Withdrawal	5000.00	2025-01-05
	TX003	ACC1002	Deposit	20000.00	2025-01-07
	TX004	ACC1003	Withdrawal	15000.00	2025-01-10
	TX005	ACC1004	Deposit	8000.00	2025-01-12
	TX006	ACC1005	Withdrawal	3000.00	2025-01-15
	TX007	ACC1006	Deposit	25000.00	2025-01-18
	TX008	ACC1007	Deposit	12000.00	2025-01-20
	TX009	ACC1008	Withdrawal	7000.00	2025-01-22
	TX010	ACC1009	Deposit	50000.00	2025-01-25
⬤	NULL	NULL	NULL	NULL	NULL

SELECT * FROM Loans;



	loan_id	customer_id	loan_type	loan_amount
▶	LN001	CUS001	Home Loan	2500000.00
	LN002	CUS003	Car Loan	800000.00
	LN003	CUS005	Education Loan	600000.00
	LN004	CUS007	Personal Loan	300000.00
	LN005	CUS010	Business Loan	1500000.00
	LN006	CUS012	Home Loan	3200000.00
*	NULL	NULL	NULL	NULL

SELECT * FROM Loan_Payments;



	payment_id	loan_id	payment_amount	payment_date
▶	PAY001	LN001	50000.00	2025-01-15
	PAY002	LN001	50000.00	2025-02-15
	PAY003	LN002	25000.00	2025-01-20
	PAY004	LN003	20000.00	2025-01-25
	PAY005	LN004	15000.00	2025-01-28
	PAY006	LN005	60000.00	2025-02-02
	PAY007	LN006	70000.00	2025-02-05
	PAY008	LN006	70000.00	2025-03-05
*	NULL	NULL	NULL	NULL

Find the customer with the highest account balance

```
SELECT c.customer_name,  
       a.balance  
FROM Customers c  
JOIN Accounts a  
ON c.customer_id = a.customer_id  
WHERE a.balance =  
(
```



```

SELECT MAX(balance)
FROM Accounts
);

```

OUTPUT

Result Grid		Filter Rows:
	customer_name	balance
▶	Ayesha	170000.00

Find the account with the minimum balance

```

SELECT *
FROM Accounts
WHERE balance =
(
    SELECT MIN(balance)
    FROM Accounts
);

```

OUTPUT

Result Grid

 Filter Rows:

Edit:   

Export 

	account_no	customer_id	branch_id	account_type	balance
▶	ACC1011	CUS011	BR005	Savings	30000.00
✱	NULL	NULL	NULL	NULL	NULL

Find customers whose balance is greater than the average balance

```

SELECT c.customer_name,
       a.balance
FROM Customers c
JOIN Accounts a

```

```

ON c.customer_id = a.customer_id
WHERE a.balance >
(
    SELECT AVG(balance)
    FROM Accounts
);

```

OUTPUT

Result Grid   Filter Rows:		
	customer_name	balance
▶	Sarah	120000.00
	Priya	98000.00
	Rahul	150000.00
	Kumar	110000.00
	Ayesha	170000.00

Find customers whose balance is less than the maximum balance

```

SELECT c.customer_name,
       a.balance
FROM Customers c
JOIN Accounts a
ON c.customer_id = a.customer_id
WHERE a.balance <
(
    SELECT MAX(balance)
    FROM Accounts
);

```

OUTPUT

Result Grid			Filter Rows:
	customer_name	balance	
▶	Ahmed	85000.00	
	John	45000.00	
	Sarah	120000.00	
	David	65000.00	
	Priya	98000.00	
	Rahul	150000.00	
	Fatima	40000.00	
	Meena	76000.00	
	Kumar	110000.00	
	Anita	53000.00	
	Ravi	30000.00	

Display the account(s) having the second highest balance

```
SELECT *  
FROM Accounts  
WHERE balance =  
(  
    SELECT MAX(balance)  
    FROM Accounts  
    WHERE balance <  
    (  
        SELECT MAX(balance)  
        FROM Accounts  
    )  
);
```

OUTPUT

	account_no	customer_id	branch_id	account_type	balance
▶	ACC1006	CUS006	BR001	Current	150000.00
*	NULL	NULL	NULL	NULL	NULL

##Find customers who have taken a loan

```
SELECT *  
FROM Customers  
WHERE customer_id IN  
(  
    SELECT customer_id  
    FROM Loans  
);
```

OUTPUT

	customer_id	customer_name	city
▶	CUS001	Ahmed	Chennai
	CUS003	Sarah	Hyderabad
	CUS005	Priya	Chennai
	CUS007	Fatima	Delhi
	CUS010	Anita	Pune
	CUS012	Ayesha	Kochi
*	NULL	NULL	NULL

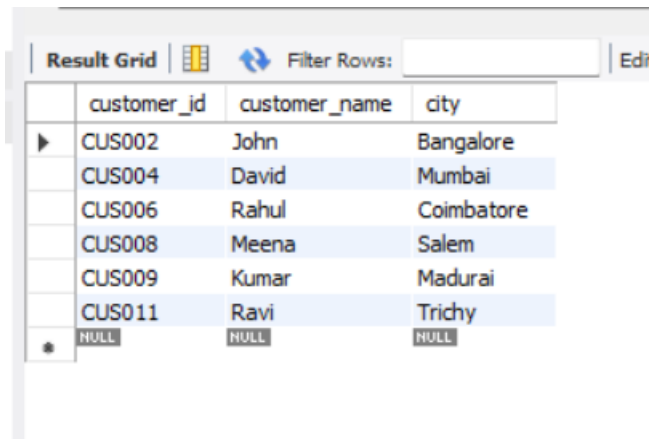
Find customers who have not taken any loan

```
SELECT *  
FROM Customers  
WHERE customer_id NOT IN  
(  
    SELECT customer_id
```

FROM Loans

);

OUTPUT



The screenshot shows a database result grid with a toolbar at the top containing 'Result Grid', a grid icon, a refresh icon, 'Filter Rows:', and an 'Edit' button. The table has three columns: 'customer_id', 'customer_name', and 'city'. It contains seven data rows and one row with three NULL values. The first row is highlighted with a mouse cursor.

	customer_id	customer_name	city
▶	CUS002	John	Bangalore
	CUS004	David	Mumbai
	CUS006	Rahul	Coimbatore
	CUS008	Meena	Salem
	CUS009	Kumar	Madurai
	CUS011	Ravi	Trichy
*	NULL	NULL	NULL

Find accounts belonging to Chennai branches

SELECT *

FROM Accounts

WHERE branch_id IN

(

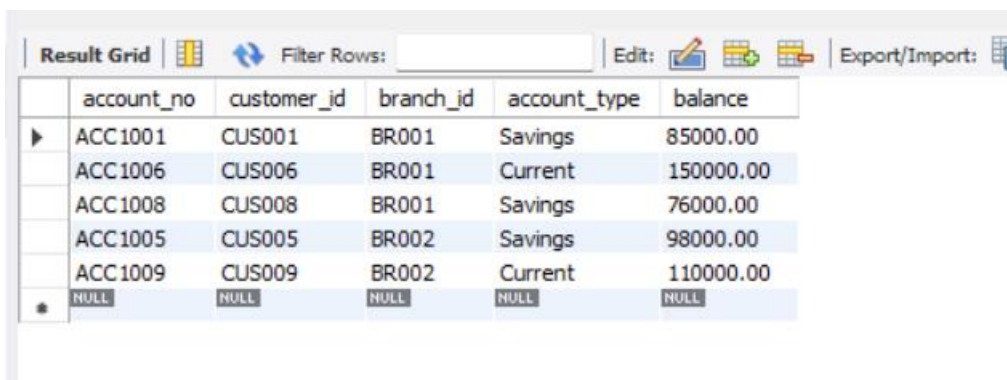
SELECT branch_id

FROM Branches

WHERE city='Chennai'

);

OUTPUT



The screenshot shows a database result grid with a toolbar at the top containing 'Result Grid', a grid icon, a refresh icon, 'Filter Rows:', 'Edit:', a pencil icon, a plus icon, a minus icon, and 'Export/Import:'. The table has five columns: 'account_no', 'customer_id', 'branch_id', 'account_type', and 'balance'. It contains five data rows and one row with five NULL values. The first row is highlighted with a mouse cursor.

	account_no	customer_id	branch_id	account_type	balance
▶	ACC1001	CUS001	BR001	Savings	85000.00
	ACC1006	CUS006	BR001	Current	150000.00
	ACC1008	CUS008	BR001	Savings	76000.00
	ACC1005	CUS005	BR002	Savings	98000.00
	ACC1009	CUS009	BR002	Current	110000.00
*	NULL	NULL	NULL	NULL	NULL

Find customers whose accounts are in Chennai branches

```
SELECT *  
FROM Customers  
WHERE customer_id IN  
(  
    SELECT customer_id  
    FROM Accounts  
    WHERE branch_id IN  
    (  
        SELECT branch_id  
        FROM Branches  
        WHERE city='Chennai'  
    )  
);
```

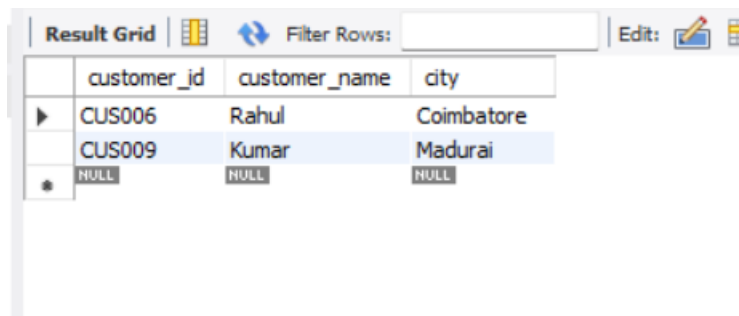
OUTPUT

Result Grid			
Filter Rows:			
Edit:			
	customer_id	customer_name	city
▶	CUS001	Ahmed	Chennai
	CUS005	Priya	Chennai
	CUS006	Rahul	Coimbatore
	CUS008	Meena	Salem
	CUS009	Kumar	Madurai
*	NULL	NULL	NULL

##Display customers who have transactions greater than ₹20,000

```
SELECT *  
FROM Customers  
WHERE customer_id IN  
(  
    SELECT customer_id  
    FROM Accounts  
    WHERE account_no IN  
    (  
        SELECT account_no  
        FROM Transactions  
        WHERE amount > 20000  
    )  
);
```

OUTPUT



	customer_id	customer_name	city
▶	CUS006	Rahul	Coimbatore
	CUS009	Kumar	Madurai
✱	NULL	NULL	NULL

Find customers whose balance is higher than the average balance of their branch

```
SELECT  
    c.customer_name,  
    a.balance,  
    b.branch_name  
FROM Customers c
```

```

JOIN Accounts a
ON c.customer_id = a.customer_id
JOIN Branches b
ON a.branch_id = b.branch_id
WHERE a.balance >
(
    SELECT AVG(a2.balance)
    FROM Accounts a2
    WHERE a2.branch_id = a.branch_id
);

```

OUTPUT

Result Grid   Filter Rows: Export: 			
	customer_name	balance	branch_name
▶	Rahul	150000.00	Chennai Main
	Kumar	110000.00	Anna Nagar
	Anita	53000.00	Bangalore Ce...
	Sarah	120000.00	Hyderabad City
	Ayesha	170000.00	Hyderabad City
	David	65000.00	Mumbai West

Find branches with more accounts than the average branch

```

SELECT
    b.branch_name,
    COUNT(a.account_no) AS TotalAccounts
FROM Branches b
JOIN Accounts a
ON b.branch_id = a.branch_id
GROUP BY
    b.branch_id,
    b.branch_name
HAVING COUNT(a.account_no) >

```



```
(
    SELECT AVG(AccountCount)
    FROM
    (
        SELECT COUNT(*) AS AccountCount
        FROM Accounts
        GROUP BY branch_id
    ) x
);
```

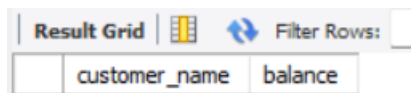
OUTPUT

Result Grid   Filter Rows:		
	branch_name	TotalAccounts
▶	Chennai Main	3
	Hyderabad City	3

Find customers whose total transaction amount exceeds their current balance

```
SELECT
    c.customer_name,
    a.balance
FROM Customers c
JOIN Accounts a
ON c.customer_id = a.customer_id
WHERE
    (
        SELECT SUM(t.amount)
        FROM Transactions t
        WHERE t.account_no = a.account_no
    ) > a.balance;
```

OUTPUT

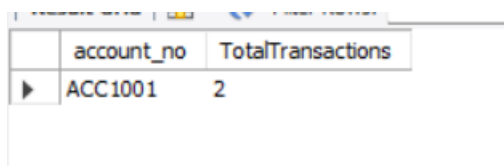


customer_name	balance
---------------	---------

Display accounts with more transactions than the average account

```
SELECT
    account_no,
    COUNT(*) AS TotalTransactions
FROM Transactions
GROUP BY account_no
HAVING COUNT(*) >
(
    SELECT AVG(TransactionCount)
    FROM
    (
        SELECT COUNT(*) AS TransactionCount
        FROM Transactions
        GROUP BY account_no
    ) x
);
```

OUTPUT



account_no	TotalTransactions
ACC1001	2

Find loan payments above the average payment for the same loan

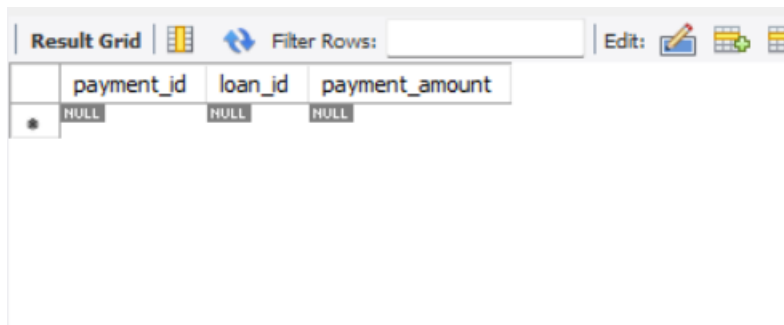
```
SELECT
    payment_id,
    loan_id,
```

```

        payment_amount
FROM Loan_Payments lp
WHERE payment_amount >
(
    SELECT AVG(payment_amount)
    FROM Loan_Payments
    WHERE loan_id = lp.loan_id
);

```

OUTPUT



	payment_id	loan_id	payment_amount
*	NULL	NULL	NULL

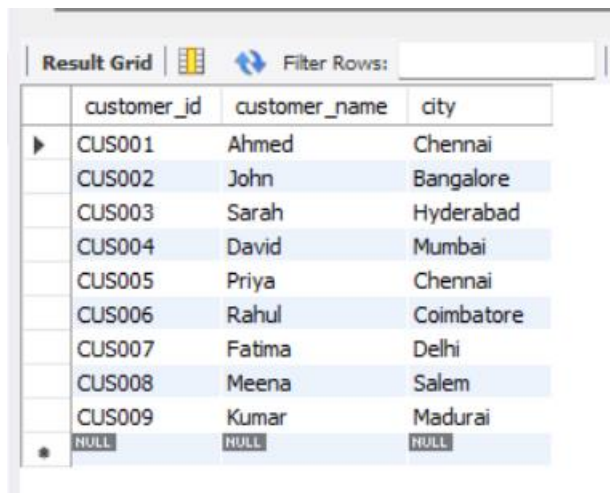
Find customers who have transactions (EXISTS)

```

SELECT *
FROM Customers c
WHERE EXISTS
(
    SELECT 1
    FROM Accounts a
    JOIN Transactions t
        ON a.account_no = t.account_no
    WHERE a.customer_id = c.customer_id
);

```

OUTPUT



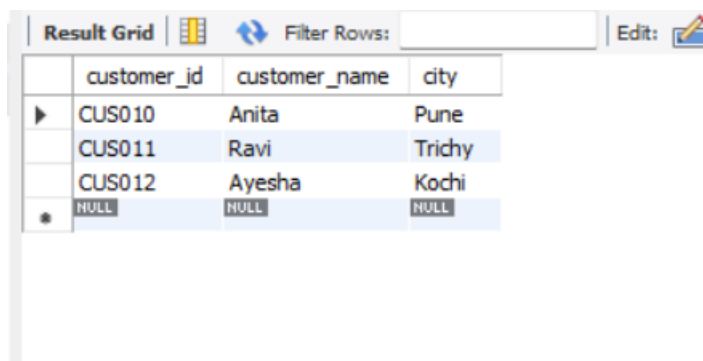
A screenshot of a database result grid. The grid has a toolbar at the top with 'Result Grid', a grid icon, a 'Filter Rows' button with a funnel icon, and a text input field. The table has four columns: 'customer_id', 'customer_name', and 'city'. There are 10 data rows, each with a blue arrow icon in the first column. The last row has 'NULL' values in all three columns and a star icon in the first column.

	customer_id	customer_name	city
▶	CUS001	Ahmed	Chennai
	CUS002	John	Bangalore
	CUS003	Sarah	Hyderabad
	CUS004	David	Mumbai
	CUS005	Priya	Chennai
	CUS006	Rahul	Coimbatore
	CUS007	Fatima	Delhi
	CUS008	Meena	Salem
	CUS009	Kumar	Madurai
*	NULL	NULL	NULL

Find customers with no transactions (NOT EXISTS)

```
SELECT *  
FROM Customers c  
WHERE NOT EXISTS  
(  
    SELECT 1  
    FROM Accounts a  
    JOIN Transactions t  
        ON a.account_no = t.account_no  
    WHERE a.customer_id = c.customer_id  
);
```

OUTPUT



A screenshot of a database result grid. The grid has a toolbar at the top with 'Result Grid', a grid icon, a 'Filter Rows' button with a funnel icon, and an 'Edit' button with a pencil icon. The table has four columns: 'customer_id', 'customer_name', and 'city'. There are 4 data rows, each with a blue arrow icon in the first column. The last row has 'NULL' values in all three columns and a star icon in the first column.

	customer_id	customer_name	city
▶	CUS010	Anita	Pune
	CUS011	Ravi	Trichy
	CUS012	Ayesha	Kochi
*	NULL	NULL	NULL

Find branches with at least one account

```
SELECT *  
FROM Branches b  
WHERE EXISTS  
(  
    SELECT 1  
    FROM Accounts a  
    WHERE a.branch_id = b.branch_id  
);
```

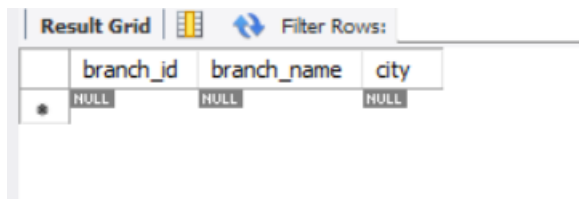
OUTPUT

Result Grid			 Filter Rows:	
	branch_id	branch_name	city	
▶	BR001	Chennai Main	Chennai	
	BR002	Anna Nagar	Chennai	
	BR003	Bangalore Ce...	Bangalore	
	BR004	Hyderabad City	Hyderabad	
	BR005	Mumbai West	Mumbai	
✱	NULL	NULL	NULL	

Find branches with no accounts

```
SELECT *  
FROM Branches b  
WHERE NOT EXISTS  
(  
    SELECT 1  
    FROM Accounts a  
    WHERE a.branch_id = b.branch_id  
);
```

OUTPUT

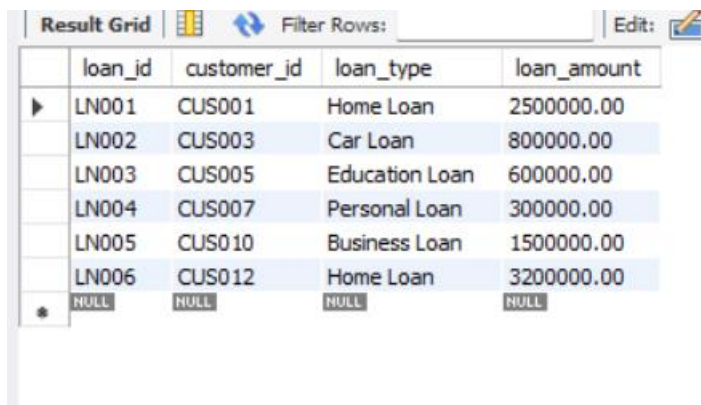


	branch_id	branch_name	city
*	NULL	NULL	NULL

Find loans that have received at least one payment

```
SELECT *  
FROM Loans l  
WHERE EXISTS  
(  
    SELECT 1  
    FROM Loan_Payments lp  
    WHERE lp.loan_id = l.loan_id  
);
```

OUTPUT



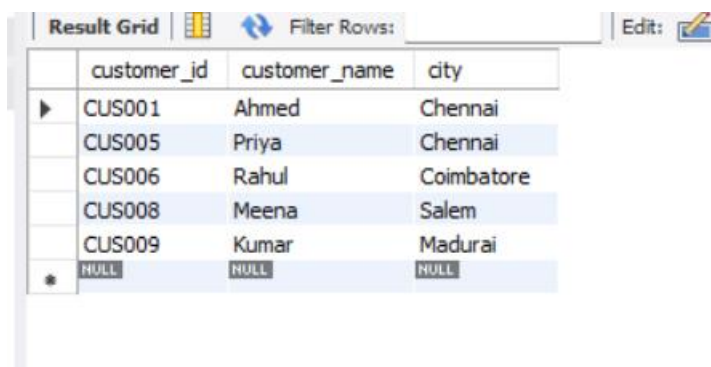
	loan_id	customer_id	loan_type	loan_amount
▶	LN001	CUS001	Home Loan	2500000.00
	LN002	CUS003	Car Loan	800000.00
	LN003	CUS005	Education Loan	600000.00
	LN004	CUS007	Personal Loan	300000.00
	LN005	CUS010	Business Loan	1500000.00
	LN006	CUS012	Home Loan	3200000.00
*	NULL	NULL	NULL	NULL

Find customers in Chennai branches using IN

```
SELECT *  
FROM Customers  
WHERE customer_id IN  
(
```

```
SELECT customer_id
FROM Accounts
WHERE branch_id IN
(
    SELECT branch_id
    FROM Branches
    WHERE city='Chennai'
)
);
```

OUTPUT



The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of a SQL query. The columns are 'customer_id', 'customer_name', and 'city'. The data rows are as follows:

customer_id	customer_name	city
CUS001	Ahmed	Chennai
CUS005	Priya	Chennai
CUS006	Rahul	Coimbatore
CUS008	Meena	Salem
CUS009	Kumar	Madurai
NULL	NULL	NULL

Find customers not having loans using NOT IN

```
SELECT *
FROM Customers
WHERE customer_id NOT IN
(
    SELECT customer_id
    FROM Loans
);
```

OUTPUT

Result Grid			
Filter Rows:			
Edit: 			
	customer_id	customer_name	city
▶	CUS002	John	Bangalore
	CUS004	David	Mumbai
	CUS006	Rahul	Coimbatore
	CUS008	Meena	Salem
	CUS009	Kumar	Madurai
	CUS011	Ravi	Trichy
✱	NULL	NULL	NULL

Find accounts with balances greater than ANY account in Mumbai

```
SELECT *  
FROM Accounts  
WHERE balance > ANY  
(  
    SELECT balance  
    FROM Accounts  
    WHERE branch_id IN  
    (  
        SELECT branch_id  
        FROM Branches  
        WHERE city='Mumbai'  
    )  
);
```

OUTPUT

Result Grid

  Filter Rows:

Edit:

Export

	account_no	customer_id	branch_id	account_type	balance
▶	ACC1001	CUS001	BR001	Savings	85000.00
	ACC1002	CUS002	BR003	Savings	45000.00
	ACC1003	CUS003	BR004	Current	120000.00
	ACC1004	CUS004	BR005	Savings	65000.00
	ACC1005	CUS005	BR002	Savings	98000.00
	ACC1006	CUS006	BR001	Current	150000.00
	ACC1007	CUS007	BR004	Savings	40000.00
	ACC1008	CUS008	BR001	Savings	76000.00
	ACC1009	CUS009	BR002	Current	110000.00
	ACC1010	CUS010	BR003	Savings	53000.00
	ACC1012	CUS012	BR004	Current	170000.00
●	NULL	NULL	NULL	NULL	NULL

##Find accounts with balances greater than ALL savings accounts

```

SELECT *
FROM Accounts
WHERE balance > ALL
(
    SELECT balance
    FROM Accounts
    WHERE account_type='Savings'
);

```

OUTPUT

Result Grid

Filter Rows:

Edit:

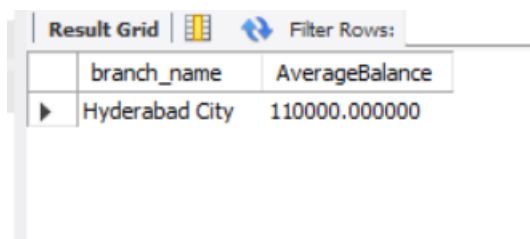
Export/Impo

	account_no	customer_id	branch_id	account_type	balance
▶	ACC1003	CUS003	BR004	Current	120000.00
	ACC1006	CUS006	BR001	Current	150000.00
	ACC1009	CUS009	BR002	Current	110000.00
	ACC1012	CUS012	BR004	Current	170000.00
*	NULL	NULL	NULL	NULL	NULL

Find branches whose average balance exceeds ALL branch averages

```
SELECT
    b.branch_name,
    AVG(a.balance) AS AverageBalance
FROM Branches b
JOIN Accounts a
ON b.branch_id = a.branch_id
GROUP BY
    b.branch_id,
    b.branch_name
HAVING AVG(a.balance) >= ALL
(
    SELECT AVG(balance)
    FROM Accounts
    GROUP BY branch_id
);
```

OUTPUT



	branch_name	AverageBalance
▶	Hyderabad City	110000.000000

Find the customer with the highest total transaction amount

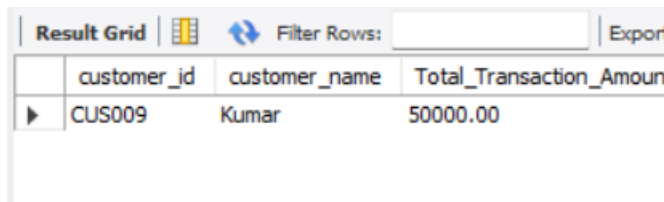
```
SELECT
    c.customer_id,
    c.customer_name,
    SUM(t.amount) AS Total_Transaction_Amount
FROM Customers c
```

```

INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
INNER JOIN Transactions t
    ON a.account_no = t.account_no
GROUP BY
    c.customer_id,
    c.customer_name
ORDER BY Total_Transaction_Amount DESC
LIMIT 1;

```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing one row of data. The table has three columns: 'customer_id', 'customer_name', and 'Total_Transaction_Amount'. The row shows 'CUS009' as the customer ID, 'Kumar' as the customer name, and '50000.00' as the total transaction amount.

	customer_id	customer_name	Total_Transaction_Amount
▶	CUS009	Kumar	50000.00

##Find customers who have both an account and a loan

```

SELECT *
FROM Customers
WHERE customer_id IN
(
    SELECT customer_id
    FROM Accounts
)
AND customer_id IN
(
    SELECT customer_id
    FROM Loans
);

```

OUTPUT

Result Grid  Filter Rows:			
	customer_id	customer_name	city
▶	CUS001	Ahmed	Chennai
	CUS003	Sarah	Hyderabad
	CUS005	Priya	Chennai
	CUS007	Fatima	Delhi
	CUS010	Anita	Pune
	CUS012	Ayesha	Kochi
*	NULL	NULL	NULL

Display the branch with the highest total deposits

SELECT

b.branch_id,

b.branch_name,

SUM(t.amount) AS Total_Deposits

FROM Branches b

INNER JOIN Accounts a

ON b.branch_id = a.branch_id

INNER JOIN Transactions t

ON a.account_no = t.account_no

WHERE t.transaction_type = 'Deposit'

GROUP BY

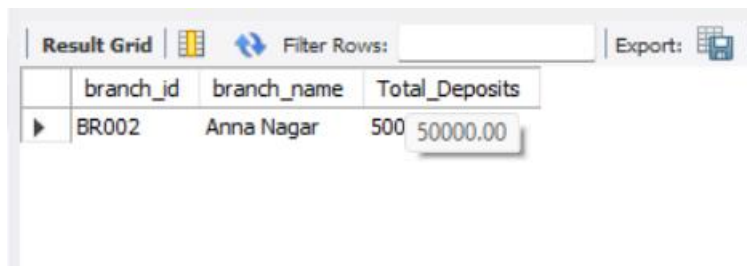
b.branch_id,

b.branch_name

ORDER BY Total_Deposits DESC

LIMIT 1;

OUTPUT



The screenshot shows a database interface with a 'Result Grid' tab. It includes a 'Filter Rows' search bar and an 'Export' button. The table has three columns: 'branch_id', 'branch_name', and 'Total_Deposits'. One row is displayed with the values 'BR002', 'Anna Nagar', and '500 50000.00'.

	branch_id	branch_name	Total_Deposits
▶	BR002	Anna Nagar	500 50000.00

##Find customers whose loan amount is above the average loan amount

SELECT

c.customer_name,

l.loan_amount

FROM Customers c

JOIN Loans l

ON c.customer_id = l.customer_id

WHERE l.loan_amount >

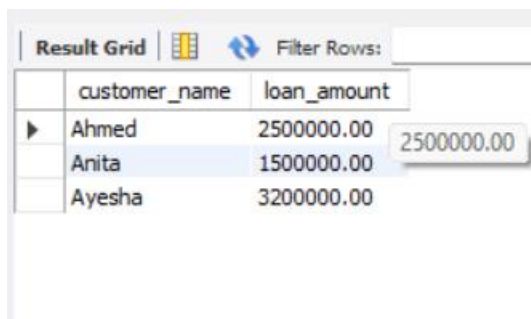
(

SELECT AVG(loan_amount)

FROM Loans

);

OUTPUT



The screenshot shows a database interface with a 'Result Grid' tab. It includes a 'Filter Rows' search bar. The table has two columns: 'customer_name' and 'loan_amount'. Three rows are displayed: 'Ahmed' with '2500000.00', 'Anita' with '1500000.00', and 'Ayesha' with '3200000.00'. The 'Anita' row is highlighted, and a tooltip shows the value '2500000.00'.

	customer_name	loan_amount
▶	Ahmed	2500000.00
	Anita	1500000.00
	Ayesha	3200000.00

Generate a customer banking report using subqueries

```
SELECT
    c.customer_name,
    c.city, (
        SELECT balance
        FROM Accounts a
        WHERE a.customer_id = c.customer_id
    ) AS AccountBalance, (
        SELECT SUM(amount)
        FROM Transactions t
        WHERE t.account_no IN
        (
            SELECT account_no
            FROM Accounts
            WHERE customer_id = c.customer_id
        )
    ) AS TotalTransactions, (
        SELECT SUM(loan_amount)
        FROM Loans l
        WHERE l.customer_id = c.customer_id
    ) AS LoanAmount
FROM Customers c;
```

OUTPUT

Result Grid					
Filter Rows:					
Export:					
Wrap Cell Content:					
	customer_name	city	AccountBalance	TotalTransactions	LoanAmount
▶	Ahmed	Chennai	85000.00	15000.00	2500000.00
	John	Bangalore	45000.00	20000.00	NULL
	Sarah	Hyderabad	120000.00	15000.00	800000.00
	David	Mumbai	65000.00	8000.00	NULL
	Priya	Chennai	98000.00	3000.00	600000.00
	Rahul	Coimbatore	150000.00	25000.00	NULL
	Fatima	Delhi	40000.00	12000.00	300000.00
	Meena	Salem	76000.00	7000.00	NULL
	Kumar	Madurai	110000.00	50000.00	NULL
	Anita	Pune	53000.00	NULL	1500000.00
	Ravi	Trichy	30000.00	NULL	NULL
	Ayesha	Kochi	170000.00	NULL	3200000.00